

Cipher Stream eXtended - CSX 1.0

Revision 1.0f, November 22, 2025

John G. Underhill - contact@qrcscorp.ca

This document is an engineering level description of the CSX encrypted and authenticated stream cipher. In its contents, a guide to implementing CSX, an explanation of its design, as well as references to its component primitives and supporting documentation.

1. Introduction

The CSX (Cipher Stream eXtended) is an authenticated stream cipher, designed for post-quantum secure communication. It incorporates a 512-bit key and uses the Keccak MAC function (KMAC) for message authentication. The cipher is optimized for both performance and security, utilizing advanced SIMD instructions and providing a flexible and tweakable initialization mechanism.

1.1 Purpose

The purpose of this specification is to define the CSX cipher in detail, describing its encryption and authentication mechanisms, and its use of the cSHAKE XOF function for input key expansion. This document also provides a security analysis of the cipher and outlines its potential applications in high-security environments such as post-quantum cryptography.

1.2 Scope

This specification covers the technical aspects of the CSX cipher, including its encryption process, authentication mechanism, key expansion, and nonce management. It also provides implementation details, including how to initialize and transform the state, the structure of the cipher, and the cryptographic functions used in its construction.

2. Cryptographic Overview

2.1 CSX Structure

The CSX cipher is a stream cipher with authenticated encryption and associated data (AEAD) functionality. The encryption rounds function was inspired by the ChaCha stream cipher, while the authentication mechanism relies on the Keccak KMAC (Keccak Message Authentication Code) for the post-quantum MAC function. The core design of the cipher incorporates the following elements:

- **Key Size:** 512 bits.
- **Nonce Size:** 128 bits (16 bytes).

- **Block Size:** 1024 bits (128 bytes).
- **Rounds:** 40 rounds of permutation.
- **Message Authentication:** KMAC-512 or a reduced round variant KMAC-R12 for faster authentication.

CSX is a stream cipher that uses a MAC function, with authenticated encryption and additional data (AEAD). The encryption uses a counter mode mechanism, where the state, nonce, and constant values are permuted and used to generate a key-stream, which is XORed with the plaintext to produce the ciphertext. The authentication process generates a MAC that is appended to the ciphertext for message integrity protection.

2.2 ChaCha-like Stream Cipher Design

CSX uses a rounds-based mixing function similar to the ChaCha stream cipher rounds function for encryption. It operates on 64-bit words, and the internal state is arranged as a 4x4 matrix of 64-bit integers. The cipher expands the key into an internal state that mixes the key, nonce, and constant values. CSX also supports vectorized implementations using AVX2 and AVX-512 instruction sets, allowing for efficient processing of multiple blocks simultaneously.

2.3 KMAC Authentication

The authentication process is built around the KMAC-512 function, a variant of the Keccak function tailored for keyed hashing. KMAC produces an authentication tag, ensuring message integrity and authenticity. In the standard mode, CSX uses KMAC-512 with 24 rounds. A reduced round variant, KMAC-R12, is also available for environments where authentication performance is prioritized over security, reducing the number of rounds in the Keccak permutation to 12 rounds. An implementation that uses just the stream cipher with no authentication is also configurable.

3. Mathematical Description

3.1 Internal State

The CSX cipher internal state is represented as a 4x4 matrix of 64-bit unsigned integers, similar to ChaCha's matrix but extended to operate with 64-bit integers and 1024-bit blocks. The state matrix is initialized with the following components:

- **Key (K):** The 512-bit secret key initialization parameter.
- **Nonce (N):** The 128-bit nonce initialization parameter.
- **Info (C):** The information string initialization parameter.

State Initialization

The state is initialized as follows:

- **Rows 0-1:** The 512-bit key is split into eight 64-bit integers, which populate the first two rows of the matrix.
- **Rows 2-3:** The constant string takes up the third row, and the nonce is split into two 64-bit integers, which populate the start of the fourth row, followed by the remainder of the constant string.

The state matrix is initialized as:

K_0	K_1	K_2	K_3
K_4	K_5	K_6	K_7
C_0	C_1	C_2	C_3
N_0	N_1	C_4	C_5

Where $K_0, K_1, \dots, K_7$ are parts of the key, N_0 and N_1 are parts of the nonce, and $C_0, C_1, C_2, C_3, C_4, C_5$ are the constant string.

3.2 ChaCha-like Permutation

CSX performs 40 rounds of a permutation operation that follows a similar structure to the ChaCha permutation. Each round consists of a series of quarter-round operations that update the state matrix. A quarter-round operation involves four elements of the matrix and performs the following steps:

1. **Addition:** Two elements are added together modulo 2^{64} .
2. **XOR:** The result is XORed with another element.
3. **Rotation:** The result is bitwise rotated by a predefined constant.

The quarter-round operation is applied to both diagonal and non-diagonal elements of the matrix to ensure a strong diffusion of input data across the state.

The rotational constants.

38	19	10	55	33	4	51	13
----	----	----	----	----	---	----	----

16	34	56	51	4	53	42	41
34	41	59	17	23	31	37	20
31	44	47	46	12	47	44	30

3.3 Key-stream Generation

After 40 rounds of mixing, the internal state is transformed into a 1024-bit key-stream block. This key-stream is XORed with the plaintext to produce the ciphertext. The 128-bit counter is incremented after each block to ensure unique key-stream blocks for every block of data.

3.4 KMAC-based Authentication

The message authentication is handled by KMAC-512. The authentication process operates as follows:

1. **Keyed Initialization:** KMAC is initialized with the user input key and optional tweak and expanded using the cSHAKE function.
2. **Message Input:** The ciphertext, nonce, encoding length, and associated data (if any) are processed by KMAC to generate the authentication tag.
3. **Tag Generation:** The finalized authentication tag is generated and appended to the ciphertext.

The authentication process ensures the integrity and authenticity of both the ciphertext and associated data.

4. Encryption and Decryption Process

4.1 Encryption Process

The encryption process in the CSX cipher follows an encrypt-then-MAC scheme, ensuring both confidentiality and authenticity of the data. The key aspects of the encryption process are described below.

4.1.1 Key Expansion

CSX uses the cSHAKE-512 function for key expansion. The 512-bit input key is processed by cSHAKE to generate both the cipher key and the authentication key. This key expansion step ensures a uniform and secure derivation of keys used in both the encryption and the MAC stages. The expanded key is divided into two parts:

- **Cipher Key:** Used in the permutation for generating the key-stream.
- **MAC Key:** Used in the KMAC-512 operation for message authentication.

This is done with two separate calls to the Keccak Squeeze function, distancing the XOF relational output between the MAC and cipher keys by a permutation call.

4.1.2 Initialization

The state matrix is initialized using the key, nonce, and info string. The 512-bit key is split into 64-bit integers, which are loaded into the first two rows of the state matrix. The first 256 bits of the constant string populate the third row, the nonce fills the first two slots of the fourth row, with the remainder of the info string populating the last two 64-bit integers of the fourth row.

4.1.3 Key-stream Generation

The key-stream is generated by applying the CSX permutation on the internal state. This permutation is repeated for 40 rounds, mixing the elements of the state matrix using addition, bitwise XOR, and rotation operations. The resulting 1024-bit output is the key-stream block, which is XORed with the plaintext to produce the ciphertext.

The key-stream generation is performed for each block of data, with the counter incremented after each block is permuted to ensure that every key-stream block is unique.

4.1.4 Encryption Steps

1. **Key Expansion:** The input key is expanded using cSHAKE-512 to produce the cipher and MAC keys.
2. **State Initialization:** The internal state is initialized with the cipher key, nonce, and constant string.
3. **Key-stream Generation:** The rounds permutation function is applied to generate the key-stream.
4. **XOR with Plaintext:** The key-stream is bitwise XORed with the plaintext to produce the ciphertext.
5. **Authentication Tag Generation:** KMAC-512 processes the ciphertext, nonce, associated data, and the encoding length, to generate the authentication tag.
6. **Append Authentication Tag:** The authentication tag is appended to the ciphertext.

4.2 Decryption Process

The decryption process steps in CSX follow the inverse of the encryption steps. The ciphertext is first authenticated, and if the authentication succeeds, the plaintext is recovered by XOR of the key-stream with the ciphertext. The key steps in the decryption process are described below.

4.2.1 Authentication Check

Before decryption, the ciphertext is authenticated using KMAC-512. The ciphertext, associated data, nonce, and encoding length (including AEAD data length) are processed by KMAC-512,

and the generated hash is compared with the hash tag appended to the ciphertext. If the tags match, the ciphertext is deemed authentic, and decryption can proceed. If the tags do not match, the decryption process is aborted and the transform function returns *false*.

4.2.2 Key-stream Generation and Decryption

Once the ciphertext is authenticated, the same key-stream generation process used in encryption is applied. The state is initialized with the same key, nonce, and constant string values, and the CSX permutation generates the key-stream. The key-stream is then XORed with the ciphertext to recover the plaintext.

4.2.3 Decryption Steps

1. **Key Expansion:** The input key is expanded using cSHAKE-512 to produce the cipher and MAC keys.
2. **State Initialization:** The internal state is initialized with the key, nonce, and constant string.
3. **Authentication Check:** The ciphertext, nonce, associated data, and encoding length, are processed using KMAC-512, and the output hash is compared to the authentication tag for equality.
4. **Key-stream Generation:** The permutation rounds function is applied to generate the key-stream.
5. **XOR with Ciphertext:** The key-stream is XORed with the ciphertext to recover the plaintext.

4.3 Performance Considerations

CSX benefits from vectorized Keccak implementations, including AVX2 and AVX512 optimizations, which allow it to process multiple blocks in parallel. These optimizations significantly improve performance on modern processors that support these instruction sets. CSX processes blocks of 1024 bits (128 bytes) at a time, and its parallel processing capabilities make it highly efficient for high-throughput encryption.

5. Mathematical Description

Where:

- k is the input cipher key
- n is the nonce
- i is the info string
- c is the output ciphertext
- ad is the additional data

- m is the input message
- tag is the MAC tag
- mk is the MAC key
- rk is the cipher key
- S is customized SHAKE
- M is KMAC
- E is the encryption function

Initialization; initialize cSHAKE with the user input key, the cipher name string or optional info tweak. The Keccak Squeeze function is called twice; once to create the cipher key, the second time to create the MAC key.

$$ck, mk = S(k \parallel i)$$

The key, nonce, and info are added to the cipher state.

$$C_{state} = \{ k, n, c \}$$

Additional Data; Additional data can be added to the MAC after initialization, and before the message is encrypted or decrypted. This data is mixed in with the MAC state, allowing authentication of related data, like a serialized packet header.

$$M_{state} = M_{mk}(ad)$$

Encryption; the cipher generates the key-stream, and XORs the key-stream with the message to produce the ciphertext.

$$c = E_{ck}(m)$$

The MAC function adds the ciphertext, nonce, and MAC input length (including AAD), to the MAC state, then calculates the MAC tag.

$$tag = M_{mk}(c \parallel n \parallel lc \parallel la)$$

The MAC tag is appended to the ciphertext.

$$c = c \parallel tag$$

Decryption; MAC the ciphertext, nonce, and MAC input length (including AAD), and compare it to the tag appended to the ciphertext.

$$tmp = M_{mk}(c \parallel n \parallel lc \parallel la)$$

If the hashes match, decrypt the ciphertext.

if $Compare(tmp, tag)$ are equal

$$m = E_{ck}(c)$$

```
else  
    return false
```

6. Implementation Guidance

This section provides implementation guidelines to help developers integrate CSX into cryptographic systems effectively.

6.1 Code Examples

Example implementations of CSX encryption and decryption using the provided functions:

Encryption

```
void csx_encrypt_example(uint8_t* enc, const uint8_t* msg, size_t msglen, const uint8_t* key, const  
uint8_t* nonce, const uint8_t* data, size_t datalen)  
{  
    qsc_csx_state ctx;  
  
    /* load the key structure */  
    qsc_csx_keyparams kp = { key, QSC_CSX_KEY_SIZE, nonce, NULL, 0 };  
  
    /* initialize the cipher state for encryption */  
    qsc_csx_initialize(&ctx, &kp, true);  
    /* add the associated data to the mac */  
    qsc_csx_set_associated(&ctx, data, datalen);  
    /* encrypt and mac the message */  
    qsc_csx_transform(&ctx, enc, msg, msglen);  
    /* dispose of the cipher state */  
    qsc_csx_dispose(&ctx);  
}
```

Decryption

```
bool csx_decrypt_example(uint8_t* msg, const uint8_t* enc, size_t enclen, const uint8_t* key, const  
uint8_t* nonce, const uint8_t* data, size_t datalen)  
{  
    qsc_csx_state ctx;  
    size_t mlen;  
    bool res;  
  
    /* load the key structure */  
    qsc_csx_keyparams kp = { key, QSC_CSX_KEY_SIZE, nonce, NULL, 0 };  
  
    /* initialize the cipher state for decryption */  
    qsc_csx_initialize(&ctx, &kp, false);  
    /* add the associated data to the mac */
```

```

    qsc_csx_set_associated(&ctx, data, datalen);
    /* mac and decrypt the message */
    mlen = enclen - QSC_CSX_MAC_SIZE;
    res = qsc_csx_transform(&ctx, msg, enc, mlen);
    /* dispose of the cipher state */
    qsc_csx_dispose(&ctx);

    return res;
}

```

7. Security Considerations

The security of the CSX cipher is based on well-established cryptographic primitives, including the design of the ChaCha stream cipher, the cSHAKE-512 function, and the KMAC-512 message authentication code. Each of these components provides a high level of cryptographic strength, making CSX a suitable choice for applications that require authenticated encryption.

7.1 Security of ChaCha Stream Cipher

ChaCha is a stream cipher designed by Daniel J. Bernstein, which has been extensively analyzed and widely adopted for its security and performance. The implementation variant used in CSX (CSX-512) operates with 40 rounds, which is considered more than sufficient to resist future cryptanalytic attacks, including those using quantum computers.

ChaCha Strengths:

- **High Performance:** ChaCha is highly optimized for performance on both software and hardware, making it suitable for a wide range of platforms.
- **Resilience Against Cryptanalysis:** ChaCha has been subjected to extensive cryptanalysis and has resisted all known attacks when configured with 20 or more rounds.
- **No Known Weaknesses:** No practical attacks have been identified against ChaCha when used with a sufficient number of rounds, ensuring its robustness for both current and future use.

7.2 Security of cSHAKE and KMAC

The cSHAKE and KMAC constructions are based on the Keccak sponge function, which forms the core of the SHA-3 standard. These constructions provide strong cryptographic guarantees, including collision resistance, pre-image resistance, and pseudorandom output.

7.2.1 cSHAKE as a Key Derivation Function (KDF)

cSHAKE is used in CSX to expand the input key into multiple keys for encryption and MAC generation. Its security is based on the same principles as SHA-3, providing resistance against brute-force attacks, key collisions, and other key-recovery attacks.

7.2.2 KMAC Security:

- **Message Authentication:** KMAC ensures the authenticity and integrity of both ciphertext and associated data. By using the KMAC-512 function, CSX provides a strong authentication mechanism that is strongly resistant to forgery attacks.
- **Security Bounds:** The 512-bit security level of KMAC ensures that it is resistant to all practical forgery and collision attacks.

7.3 Resistance to Side-Channel Attacks

CSX is designed to minimize its susceptibility to side-channel attacks, such as timing attacks and power analysis. This is achieved through the following strategies:

- **Constant-Time Operations:** The core operations of CSX are designed to execute in constant time, preventing attackers from gaining insights into the key or internal state through timing variations.
- **Minimal Data Leakage:** Sensitive information, such as key material and intermediate states, is securely erased from memory as soon as it is no longer needed, reducing the risk of leakage through memory analysis.

7.4 Protection Against Quantum Attacks

As cryptographic research progresses, quantum computers pose a serious threat to many classical cryptographic algorithms. While symmetric key ciphers like ChaCha are less vulnerable to quantum attacks than asymmetric algorithms, CSX incorporates forward-thinking design choices to mitigate quantum risks:

- **Increased Key Size:** The 512-bit key size of CSX provides a higher security margin, making it more resistant to attacks from quantum computers compared to smaller key sizes, thus restoring the security margins to a modern symmetric cipher.
- **Post-Quantum Security Awareness:** By relying on Keccak-based primitives (cSHAKE and KMAC), CSX inherits the post-quantum resistance characteristics of these constructions, which are considered secure against quantum cryptanalysis.

7.5 Authentication and Integrity Guarantees

The authenticated encryption mechanism used by CSX ensures both confidentiality and integrity of the encrypted data. The following guarantees are provided:

- **Confidentiality:** The CSX stream cipher ensures that the plaintext remains confidential, even in the presence of an attacker with access to the ciphertext.
- **Integrity:** The KMAC-512 function provides strong integrity guarantees, ensuring that any modifications to the ciphertext or associated data are detected and rejected during decryption.

- **Authenticity:** Only someone with access to the correct key can produce a valid ciphertext and MAC, ensuring the authenticity of the sender.

8. Cryptographic Analysis

8.1 Security Objectives

Objective	Target property	Metric	CSX claim
Confidentiality	IND-CCA	$\geq 2^{\kappa/2}$ time (Grover bound)	$\kappa = 512\text{-bit key} \Rightarrow \geq 2^{256}$ effort
Integrity	INT-CTXT	$\leq 2^{-\lambda}$ tag-forge prob. (Q2)	$\lambda = 512$ (KMAC-512)
Replay resistance	Unique nonce	Protocol layer	128-bit nonce, developer-enforced uniqueness

8.2 Design Rationale

- **Wide-word ChaCha core.**
Up-sizing ChaCha to 64-bit words doubles per-round diffusion and raises the birthday bound to 2^{128} blocks; forty rounds deliver a $2 \times$ margin over the original 20-round design.
- **cSHAKE key separation.**
Two consecutive squeezes of cSHAKE-512 generate disjoint cipher and MAC keys, eliminating related-key cross-talk and giving domain separation CSX Specification.
- **KMAC authentication.**
The keyed-sponge MAC inherits Keccak-f[1600]’s 24-round safety margin; no hidden-period attacks such as Simon apply to KMAC.
- **Round count (40).**
Current distinguishers fail beyond 10-round 32-bit ChaCha; the latest record reaches 7.5 rounds with 2^{253} time . Scaling to 64-bit words and quadrupling the round budget gives ≥ 30 -round headroom.

8.3 Classical Cryptanalysis

Attack family	Best public complexity†	CSX-512 status	Main sources
Exhaustive key search	2^{512}	Infeasible	—

Differential / linear	$\geq 2^{300}$ data	> 30 -round margin	
Rotational/XOR bias	None > 8 rounds	No trail across 40 rounds	CSX Specification
Related-key	Needs key-mixing weakness	Absent (cSHAKE keys)	CSX Specification
Tag forgery	$q / 2^{512}$	2^{-512} per try	

† Full-cipher attacks; reduced-round distinguishers excluded.

Key takeaway. All published ChaCha-style attacks hinge on reduced rounds or 32-bit word structure—neither applies to CSX. No structural shortcut undermines KMAC if capacity ≥ 512 bits.

8.4 Quantum-Adapted Analysis

Quantum primitive	Effect	Residual cost	Notes
Grover search	$\sqrt{\text{speed-up on brute-force}}$	2^{256} trials	Meets NIST Cat-V guidance
BHT collision	Wide-block 1024-bit stream	2^{128} blocks	Generic lower bound
Period-finding (Simon)	Breaks GMAC/Poly1305	Not applicable	No algebraic period in KMAC sponge
Hidden-shift on Keccak	Distinguish ≤ 12 rounds	None for 24-round KMAC	[Grover]

8.5 Side-Channel & Fault Model

- **Constant-time reference.** CSX avoids table look-ups and key-dependent branches for both permutation and KMAC processing.
- **Vectorised transforms.** AVX2/AVX-512 paths seal timing leakage and reach > 4 GiB/s on Ice-Lake-SP (vendor benchmark) CSX Specification.
- **Masking feasibility.** Boolean first-order masking increases latency $\approx 1.7\times$; code comments outline share-refresh hooks.
- **Fault detection.** Final KMAC tag verification provides immediate fault-injection detection with probability $\geq 1 - 2^{-512}$.

8.6 Security Margin

- **Differential probability.** Any 5-round trail on 64-bit ChaCha has prob. $\leq 2^{-260}$; forty rounds leave ≥ 35 -round slack.
- **Linear bias.** Maximum bias $\varepsilon \leq 2^{-130}$ (derived from 32-bit ChaCha bias scaled by 2-word mixing).
- **Nonce misuse.** Duplicate (K, N) leaks plaintext XOR and invalidates authenticity. 128-bit nonces give a 2^{64} birthday margin; implementers **must** prevent reuse.

8.7 AEAD Composition Proof Sketch

CSX employs *Encrypt-then-MAC* with a strongly unforgeable, collapsing MAC; Bellare–Namprempre show this gives IND-CCA and INT-CTXT security in the classical model, and Gagliardoni et al. extend the result to the quantum world for capacity $\geq 2\lambda$, satisfied here ($c = 512 > 2 \times \lambda$).

8.8 Comparative Snapshot

Property	CSX-512 + KMAC	ChaCha20-Poly1305
Word / block	64 bit / 1024 bit	32 bit / 512 bit state
Rounds	40	20
Key size	512 bit	256 bit
MAC	KMAC-512 (32–64 B tag)	Poly1305 (16 B)
Q2 tag security	256 bit	64 bit
HW acceleration	AVX2/AVX-512, SHA-3	AVX2

CSX trades a larger footprint for double-tag security and wider diffusion.

8.9 Summary

No known classical or quantum attack reduces CSX below its claimed 256-bit security level under unique-nonce usage. The 40-round, 64-bit permutation, cSHAKE key separation, and KMAC-512 authentication collectively deliver a comfortable margin. Remaining risk vectors are nonce reuse and implementation leakage—mitigations are prescribed in §7 of the specification.

Citation Table

Tag	Title / Source	URL
[SPC]	CSX Specification v1.0c (internal project paper)	— (private document uploaded by you; no public link) CSX Specification
[Cha24]	Al-Dey <i>et al.</i> “Improved Cryptanalytic Results against ChaCha” (IACR ePrint 2025/428, PDF)	https://eprint.iacr.org/2025/428.pdf eprint.iacr.org
[KMAC]	NIST SP 800- 185 “SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, ParallelHash”	https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-185.pdf nvlpubs.nist.gov
[BN00]	Bellare & Namprempre “Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm” (IACR ePrint 2000/025)	https://eprint.iacr.org/2000/025 eprint.iacr.org
[Grover]	Scott Aaronson blog post “Of course Grover’s	https://scottaaronson.blog/?p=7143 scottaaronson.blog

	algorithm offers a quantum advantage!” (Mar 22 2023)	
[SHA3- SM]	Jian Guo “Security of SHA-3 and Related Constructions ” (FSE 2019 slides, PDF)	https://fse.iacr.org/2019/files/slides/fse2019_slides-guo.pdf